ChatGPT

In-depth Evaluation of the Web-Search MCP

1 Context and scope

The web-search MCP is a FastMCP server that exposes tools for multi-provider web search (`web_search`), page extraction (`get_content` / `batch_get_content`) and generative search (`gemini_search`, `perplexity_search`), with optional YouTube search/transcripts and Composio queries. It aggregates results from SearXNG, DuckDuckGo, Tavily, Brave, Jina and other providers, merges them via Weighted **Reciprocal Rank Fusion (RRF)**, and optionally reranks them using a bi-encoder and cross-encoder pipeline. The server also caches queries (exact and semantic) and page content to optimise latency and cost.

This report focuses solely on the web-search MCP. It evaluates the design and implementation, identifies gaps or suboptimal areas, and suggests quick wins and longer-term improvements. Citations are provided from external sources on retrieval strategies and search algorithms where relevant.

2 Detailed assessment of existing functionality

2.1 Search orchestration

1. **Provider selection and query rewriting.**

2. The `web_search` tool accepts a `query`, a mandatory `research_goal`, and optional `providers`. By default, SearXNG, DuckDuckGo and Gemini fire automatically, while paid providers (Brave, Tavily, Jina, Composio LLM search) are only called when explicitly requested. This conditional model protects budgets.

3. **Query rewriting** is enabled by default. The server uses a light LLM to generate variants of the user query and selects the best ones to run; these variants are then processed in parallel. This aligns with recommended retrieval strategies that suggest using LLMs to **rewrite** or **expand** user queries to correct spelling, clarify ambiguities and add synonyms [1]. However, details of the rewrite algorithm (prompt design, number of variants, weighting) are not exposed.

4. **Result merging and reranking.**

5. Search results from each provider are merged via Weighted RRF. RRF combines results based on their ranks rather than raw scores, making it robust to providers with incompatible scoring scales [2]. The server uses a fixed `k` value and optionally applies weights per provider.

6. After merging, a multi-stage reranking pipeline runs if more than one result is returned: (i) a bi-encoder filters candidates down to ~2× `top_k` based on embedding similarity, (ii) Jina AI's cross-encoder reranks the filtered list, and (iii) maximal marginal relevance (MMR) removes near duplicates and increases topical diversity. These steps are consistent with recommended reranking

approaches: cross-encoders offer high precision at the cost of latency, while diversity pruning reduces redundancy [3].

7. **Caching and singleflight.**

8. An exact query cache stores responses keyed on query, number of results, rewrite flag and provider list. A semantic cache stores embeddings of queries and results with adaptive TTL based on content type. This is beneficial because repeated queries or similar queries can be served quickly from cache.

9. The server uses a **singleflight** mechanism to coalesce concurrent identical queries, reducing duplicate API calls during high concurrency.

10. **Middleware guidance and controls.**

11. Query and result guidance middlewares add non-blocking tips to help agents formulate better searches and interpret results. Expensive tool protection middleware encourages a "cheapest first" strategy, preventing premature calls to Perplexity or other high-cost providers.

12. Rate-limit middleware differentiates between cheap and expensive providers, controlling API usage.

2.2 Content extraction and summarisation

1. **Page fetching backends.** The `get_content` and `batch_get_content` tools use multiple backends: `safe_http_extract` for simple pages, `jina_reader` for summarising with Jina AI, `browser_fallback` using a headless Chrome pool for dynamic pages, plus specialised extractors for ArXiv, GitHub issues/discussions, StackExchange and YouTube transcripts. Extractors return structured metadata (`status`, `source_type`, `fetch_backend`, `page_content`, pagination window). This variety helps handle diverse content types but also introduces complexity.

2. **Summaries and windows.** The `summary_mode` parameter in `get_content` requests a summarised version (brief/detailed) using the "Chutes" API, and `focus_query` can guide the summary. `batch_get_content` supports sliding windows through `char_offset` and `char_length`, enabling progressive reading of long documents. These features are valuable but require additional API keys and can be slow.

3. **YouTube and generative search.** The server includes `youtube_search` and `youtube_transcript` tools. Generative search tools (`gemini_search` and `perplexity_search`) provide natural language answers with citations, but are gated to prevent overuse. They offload synthesis to external LLMs rather than combining search results internally.

2.3 Strengths

- **Robustness through multi-provider aggregation.** Combining multiple search providers reduces reliance on a single API and improves coverage. Weighted RRF leverages consensus across providers to surface reliable results [2]. The system also handles provider failures gracefully.

- **Advanced reranking.** The two-stage reranking pipeline (bi-encoder + cross-encoder + diversity) follows best practices in retrieval pipelines [3] and improves the quality of the top results.

- **Caching and concurrency.** Layered caching and singleflight reduce latency and API calls. Adaptive TTL in the semantic cache balances freshness and cost.

- **Middleware guidance.** Guidance middlewares encourage agents to refine queries and think before using expensive tools, aligning with cost-effective agent design.

- **Extensible architecture.** Providers are pluggable. Additional providers could be added simply by implementing a client and registering it. Tools are annotated via FastMCP, and provider selection is configurable at call time.

3 Gaps and sub-optimal solutions

Despite its strengths, the web-search MCP has several areas that could be improved.

3.1 Limited transparency and control over query rewriting

- **Hidden rewrite logic.** Agents cannot inspect or modify how queries are rewritten. The server might generate multiple variants that the agent would consider irrelevant or overly broad. For example, a coding agent searching for "TypeError: cannot read property of undefined" might have the error truncated or paraphrased, losing its exact context. Retrieval guides emphasise that rewriting should preserve original meaning and add synonyms while correcting spelling [1]. Without transparency, agents cannot decide whether rewriting helped or harmed the results.

- **Lack of multi-query retrieval options.** The rewriting step appears to produce a single best query or a few variants but does not expose a multi-query retrieval mode in which all variants are run and merged. Multi-query retrieval is recommended for generating comprehensive sets of documents [1]. Agents might want to run the original query and the rewrites separately.

3.2 Provider weighting and dynamic selection

- **Fixed weights and order.** Weighted RRF is used, but weights for each provider are hard-coded and not exposed as configuration or call parameters. Query types vary: coding errors may be better served by Google/Brave, whereas general knowledge may benefit from Jina. Without dynamic weighting or query-dependent provider selection, the system may favour or penalise providers incorrectly.

- **Lack of provider reliability metrics.** Providers vary in freshness, coverage and failure rates. The orchestrator does not appear to track per-provider success rates, response times or error patterns to adjust future selections. This can lead to repeated calls to unstable providers instead of routing queries to more reliable ones.

3.3 Result information limitations

- **Missing provider consensus signals.** The docstring mentions that `provider_count` can serve as an agreement signal, but this field is not returned to clients. Agents would benefit from knowing how many providers returned each result, as higher agreement can indicate reliability.

- **No metadata on publication date or language.** Results include title, link and snippet but not publication date, domain category or detected language. Without these, agents cannot easily filter out outdated or non-English results.

- **Limited deduplication across providers.** While the reranking stage includes MMR, the merging step can still produce near duplicates (e.g., the same article syndicated across multiple domains). Additional deduplication heuristics could be applied before or after merging.

3.4 Generative tools and summarisation gaps

- **Siloed generative search.** Generative search tools (Gemini and Perplexity) return synthesized answers with citations, but they operate independently of the classical search pipeline. There is no unified tool that first runs a normal search, fetches page content and then uses an LLM to summarise the findings. Agents must orchestrate these steps manually.

- **Reliance on external summarisation APIs.** `summary_mode` uses “Chutes” API (likely an internal summariser) and `jina_reader` uses Jina AI. There is no fallback summarisation using local models. If API keys are missing or the service fails, summarisation is unavailable.

3.5 Content extraction challenges

- **Complex backend selection logic.** The fetch pipeline chooses between `safe_http_extract`, `jina_reader`, `browser_fallback` and other specialised extractors. However, the criteria for choosing each backend are not transparent, and there is no mechanism for the agent to hint which backend to use. Sometimes a page might be scraped with a simpler extractor when a headless browser would have yielded richer content (e.g., dynamic websites).

- **Limited control over concurrency and timeouts.** The environment variables `KINDLY_WEB_SEARCH_MAX_CONCURRENCY` and `KINDLY_TOOL_TOTAL_TIMEOUT_SECONDS` affect concurrency and time budgets, but they are global. Agents cannot request more or fewer concurrent fetches or adjust per-call timeout thresholds.

3.6 Diagnostic and observability limitations

- **No tool for retrieving telemetry.** The server collects metrics on provider usage, cache hits and latencies, but there is no public tool to retrieve this information. Agents could make smarter decisions (e.g., avoid providers that are currently failing) if they had access to aggregated stats.

- **Verbosity control.** Guidance middlewares always emit messages. For some agent contexts (e.g., automated pipelines), it might be beneficial to suppress or reduce guidance verbosity. There is no API to adjust this at call time.

4 Quick wins (incremental improvements)

1. **Expose provider count and source domain in results.** Add fields such as `provider_count`, `providers_used` and `domain` to each search result so agents can prioritise links with stronger consensus. Also include `published_date` and `language` when they can be extracted from the search provider or page metadata.

2. **Add a `rewritten_queries` field to the `web_search` response.** Return the rewritten versions of the query (if rewriting was enabled) and indicate which variant produced each result. This improves transparency and helps agents debug or fine-tune queries. If rewriting is disabled, return an empty list.

3. **Allow per-call provider weighting.** Extend the `providers` parameter to accept weights or priorities, e.g., `{"brave": 2.0, "ddg": 0.5}`. When not specified, fall back to default weights. This gives agents more control over which providers dominate the final ranking.

4. **Return guidance messages separately.** Add an optional flag (e.g., `verbose: bool`) that controls whether query and result guidance tips are returned. This prevents clutter in contexts where the agent already knows how to use the tool.

5. **Offer a simpler `search_and_fetch` helper.** Provide a tool that performs `web_search` and then automatically fetches the first result's content (or a configurable number of top results) using `get_content`, returning both the search metadata and the fetched pages. This reduces the number of round-trips for common patterns such as "search then read".

6. **Include fallback summarisation.** When `summary_mode` is requested but external summarisation services are unavailable, use a locally hosted small LLM (e.g., a distilled summariser) to generate summaries. Agents benefit from consistent behaviour without requiring extra API keys.

7. **Improve deduplication in merging.** After Weighted RRF, cluster results by URL fingerprint or content snippet to detect near duplicates and keep only the best representative of each cluster. This reduces redundancy in `web_search` results.

8. **Expose diagnostics information.** Add a tool (e.g., `get_search_metrics`) that returns recent statistics such as provider error rates, average latency, cache hit rates and reranking time. Agents can use this to choose providers or adjust timeouts.

9. **Parameterise concurrency and timeouts per call.** Allow `web_search` and `batch_get_content` to accept optional `max_concurrency` and `timeout` arguments, clamped within safe bounds. This enables agents to trade off latency versus completeness when necessary.

5 Longer-term opportunities and feature ideas

1. **Multi-query retrieval and decomposition.** Implement an optional mode in which the LLM generates multiple diverse queries (multi-query retrieval) or decomposes complex questions into

sub-queries [4]. Run each query against providers and merge the results via RRF or other fusion methods. Provide the list of sub-queries and a breakdown of which results came from each. This yields richer result sets and is especially useful for complex research questions.

2. **Adaptive provider routing.** Track per-provider success rates, latency, and domain specialisation over time. Use these metrics to dynamically route queries. For example, if Brave becomes unresponsive, temporarily reduce its weight; if Jina consistently returns relevant results for code-related queries, prioritise it for such queries. Provide a feedback mechanism for agents to rate results, feeding into future routing decisions.

3. **Domain-aware and topic-aware search.** Add a classifier that analyses the query and research goal to infer the desired domain (e.g., programming, academic, news, consumer). Use this to adjust provider weighting and result reranking. For instance, for research queries, favour Jina and Brave; for programming queries, favour providers that index StackOverflow or GitHub discussions.

4. **Interactive query refinement.** Create a conversational helper tool that, instead of automatically rewriting the query, returns suggestions for refinement based on query analysis and the research goal. The agent can choose a suggestion or provide more context. This interactive approach prevents over-rewriting and engages the user in the search process.

5. **RAG-style summarised answer generation.** Provide a single tool that runs a balanced search, fetches the top-k pages, extracts their content, and uses a local or external LLM to produce a concise, cite-backed answer. This would be a high-level alternative to generative search tools like Perplexity but with more transparency and reproducibility. It should include the ability to control answer length and number of sources.

6. **Integration with structured and relational data.** Many queries involve structured data (IDs, dates, codes). Consider adding an entity detector that recognises structured fields in queries and routes them to the appropriate backend (e.g., a SQL database or API). This aligns with recommended hybrid retrieval strategies that use metadata filtering and structured lookups for exact matches [5].

7. **Translation and language awareness.** Detect the user's language and allow specifying the desired language for results. Integrate translation of snippets or full pages when necessary. This would make the MCP useful for non-English users or when retrieving foreign content.

8. **User-configurable personas and search modes.** Expose different search modes— `fast`, `balanced`, `deep` —that control which providers fire, how many rewritten queries are generated, and whether reranking runs. Users could also define custom personas (e.g., "academic researcher", "developer", "journalist"), each with tailored provider configurations and prompts for rewriting.

9. **Improved extraction heuristics.** Provide an API for agents to hint which extraction backend to use (e.g., always use the browser for dynamic sites, use PDF extractor for PDF links). Make extractor selection pluggable so that new extractors can be added or existing ones improved without modifying core code.

10. **Result clustering and topic extraction.** After merging results, cluster them by topic using embeddings and identify prominent themes. Return clusters with representative links and short descriptions. This helps agents navigate results and plan next steps.

6 Conclusion

The web-search MCP is a powerful and well-architected tool for multi-provider search, merging and content extraction. It follows many best practices—multi-provider aggregation, Weighted RRF fusion, cross-encoder reranking, caching and middleware guidance—and provides a solid foundation for agentic information retrieval. However, several aspects can be improved: transparency of query rewriting, flexibility in provider weighting, richer result metadata, unified generative summarisation and better diagnostic exposure. Quick wins such as exposing provider counts, rewritten queries and adding per-call overrides can be implemented with minimal risk. Longer-term features like multi-query retrieval, adaptive routing and integrated RAG answers could significantly enhance the server's value and usability for diverse agent workflows.

---

1 3 4 5 Retrieval strategies: Finding the right information | Anyscale Docs
https://docs.anyscale.com/rag/quality-improvement/retrieval-strategies

2 Reciprocal Rank Fusion: the one-line algorithm behind hybrid search | Serghei's Blog
https://blog.serghei.pl/posts/reciprocal-rank-fusion-explained/